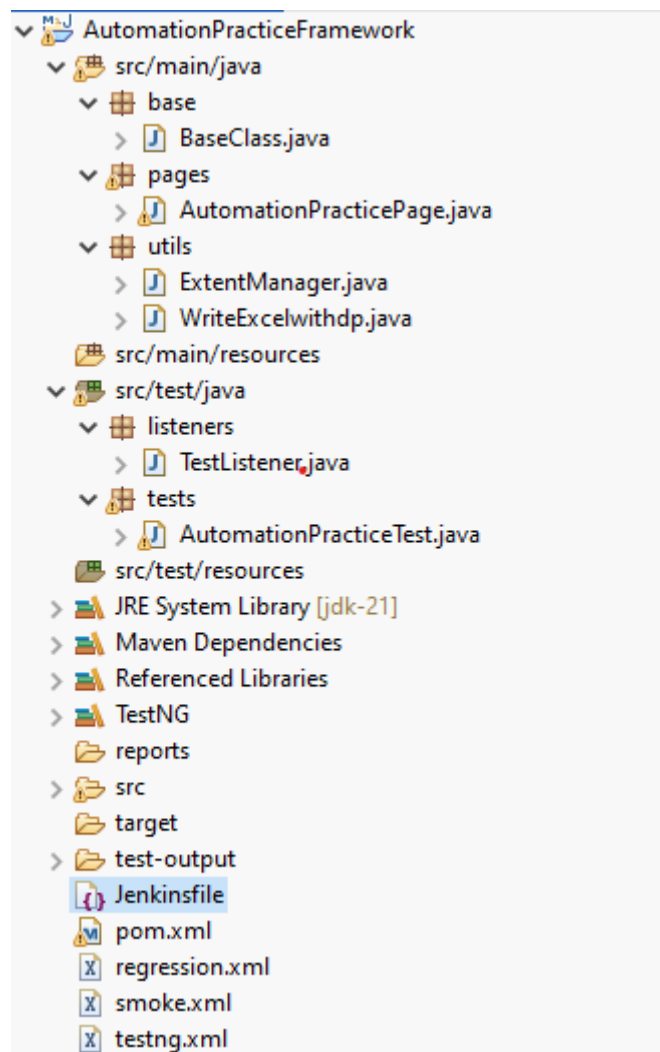


AUTOMATION TESTING PRACTICE FRAMEWORK

The AutomationPracticeFramework is a well-structured Java-based automation testing project built with Maven and TestNG, designed to support scalable test execution and reporting. It follows a layered architecture where base handles browser setup and teardown, pages implements Page Object Models for reusable UI interactions, utils provides supporting classes like Excel data handling and ExtentReports configuration, and listeners manage test execution events for logging and reporting. Test classes reside under tests, while resources, reports, screenshots, and configuration files such as testng.xml, smoke.xml, and regression.xml organize execution flows. CI/CD integration is enabled through a Jenkinsfile, and dependencies are managed via pom.xml, ensuring maintainability, reusability, and professional reporting of automated test results.

Project Structure:



BaseClass.Java

```
package base;

import java.time.Duration;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

import io.github.bonigarcia.wdm.WebDriverManager;

public class BaseClass {

    public static WebDriver driver;

    public void setup() {

        WebDriverManager.chromedriver().setup();

        driver = new ChromeDriver();

        driver.manage().window().maximize();

        driver.manage().timeouts()
            .implicitlyWait(Duration.ofSeconds(10));

        driver.get("https://testautomationpractice.blogspot.com/");
    }

    public void tearDown() {

        if(driver != null) {

            System.out.println();
            System.out.println("=====");
            System.out.println(" CLOSING BROWSER...");
            System.out.println("=====");

            driver.close();

            driver = null;
        }
    }
}
```

AutomationPracticePage.Java

```
package pages;
```

```
import java.io.File;

import java.time.Duration;

import java.util.List;

import java.util.Set;

import org.openqa.selenium.By;

import org.openqa.selenium.JavascriptExecutor;

import org.openqa.selenium.SearchContext;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.support.ui.ExpectedConditions;

import org.openqa.selenium.support.ui.Select;

import org.openqa.selenium.support.ui.WebDriverWait;

public class AutomationPracticePage {

    WebDriver driver;

    public AutomationPracticePage(WebDriver driver) {

        this.driver = driver;

    }

    // Utility: Auto-scroll and type

    public void enterFieldWithAutoScroll(By locator, String value) {

        WebElement element = driver.findElement(locator);

        ((JavascriptExecutor) driver).executeScript(

            "arguments[0].scrollIntoView({behavior: 'smooth', block: 'center'});", element);

        try { Thread.sleep(300); } catch (Exception e) {}

        element.clear();

        element.sendKeys(value);

    }

    // UNIT 1 - GUI ELEMENTS

    By txtName = By.id("name");

    By txtEmail = By.id("email");
```

```
By txtPhone = By.id("phone");
```

```
By txtAddress = By.id("textarea");
```

```
By maleRadio = By.id("male");
```

```
By femaleRadio = By.id("female");
```

```
By sunday = By.id("sunday");
```

```
By monday = By.id("monday");
```

```
By tuesday = By.id("tuesday");
```

```
By countryDropdown = By.id("country");
```

```
By colorDropdown = By.id("colors");
```

```
By animalDropdown = By.id("animals");
```

```
public void enterName(String name) {  
    driver.findElement(txtName).clear();  
    driver.findElement(txtName).sendKeys(name);  
    System.out.println("RESULT: Name entered successfully.");  
}
```

```
public void enterEmail(String email) {  
    driver.findElement(txtEmail).clear();  
    driver.findElement(txtEmail).sendKeys(email);  
    System.out.println("RESULT: Email entered successfully.");  
}
```

```
public void enterPhone(String phone) {  
    driver.findElement(txtPhone).clear();  
    driver.findElement(txtPhone).sendKeys(phone);  
    System.out.println("RESULT: Phone entered successfully.");  
}
```

```
public void enterAddress(String address) {  
    driver.findElement(txtAddress).clear();  
    driver.findElement(txtAddress).sendKeys(address);  
    System.out.println("RESULT: Address entered successfully.");  
}
```

```
public void selectGender(String gender) {  
    if (gender.equalsIgnoreCase("Male"))  
        driver.findElement(maleRadio).click();  
    else  
        driver.findElement(femaleRadio).click();  
    System.out.println("RESULT: Gender selected: " + gender);  
}
```

```
public void selectDays() {  
    driver.findElement(sunday).click();  
    driver.findElement(monday).click();  
    driver.findElement(tuesday).click();  
    System.out.println("RESULT: Days selected: Sunday, Monday, Tuesday");  
}
```

```
public void selectCountry(String country) {  
    new Select(driver.findElement(countryDropdown)).selectByVisibleText(country);  
    System.out.println("RESULT: Country selected: " + country);  
}
```

```
public void selectColor(String color) {  
    new Select(driver.findElement(colorDropdown)).selectByVisibleText(color);  
    System.out.println("RESULT: Color selected: " + color);  
}
```

```
public void selectAnimal(String animal) {  
    new Select(driver.findElement(animalDropdown)).selectByVisibleText(animal);  
    System.out.println("RESULT: Animal selected: " + animal);  
}
```

```
// =====
```

```
// UNIT 2 - DATE PICKERS
```

```
// =====
```

```
By datePicker1 = By.id("datepicker");  
By startDate = By.id("start-date");  
By endDate = By.id("end-date");  
By submitDateBtn = By.xpath("//button[text()='Submit']");
```

```
public void selectDatePicker1(String date) {  
    driver.findElement(datePicker1).clear();  
    driver.findElement(datePicker1).sendKeys(date);  
    driver.findElement(By.tagName("body")).click();  
    System.out.println("RESULT: Date selected in DatePicker1: " + date);  
}
```

```
public void selectDatePicker2(String date) {  
    ((JavascriptExecutor) driver).executeScript(  
        "document.getElementById('txtDate').value='" + date + "';");  
    System.out.println("RESULT: Date selected in DatePicker2: " + date);  
}
```

```
public void selectDateRange(String start, String end) {  
    driver.findElement(startDate).clear();  
    driver.findElement(startDate).sendKeys(start);  
    driver.findElement(endDate).clear();
```

```

        driver.findElement(endDate).sendKeys(end);

        System.out.println("RESULT: Date range selected: " + start + " to " + end);
    }

    public void submitDateRange() {
        ((JavascriptExecutor) driver).executeScript(
            "arguments[0].click();", driver.findElement(submitDateBtn));
        System.out.println("RESULT: Date range submitted successfully.");
    }

// =====
// UNIT 3 - FILE UPLOAD
// =====

By singleFile = By.id("singleFileInput");
By multipleFiles = By.id("multipleFilesInput");

    public void uploadSingleFile(String path) {
        driver.findElement(singleFile).sendKeys(path);
        System.out.println("RESULT: Single file uploaded: " + path);
    }

    public void uploadMultipleFiles(String file1, String file2) {
        driver.findElement(multipleFiles).sendKeys(file1 + "\n" + file2);
        System.out.println("RESULT: Multiple files uploaded: " + file1 + ", " + file2);
    }

// =====
// UNIT 4 - PAGINATION WEB TABLE
// =====

By pages = By.xpath("//ul[@id='pagination']/li");
By tableRows = By.xpath("//table[@id='productTable']/tbody/tr");

```

```

public int getTotalPages() {
    return driver.findElements(pages).size();
}

public void selectAllProductsInAllPages() throws InterruptedException {
    List<WebElement> totalPages = driver.findElements(pages);
    int pageCount = totalPages.size();
    JavascriptExecutor js = (JavascriptExecutor) driver;

    for (int p = 1; p <= pageCount; p++) {
        if (p > 1) {
            driver.findElement(By.xpath("//ul[@id='pagination']/li[" + p + "]")).click();
        }

        List<WebElement> rows = driver.findElements(tableRows);
        for (int r = 1; r <= rows.size(); r++) {
            String product = driver.findElement(
                By.xpath("//table[@id='productTable']/tbody/tr[" + r + "]/td[2]")).getText();
            String price = driver.findElement(
                By.xpath("//table[@id='productTable']/tbody/tr[" + r + "]/td[3]")).getText();

            System.out.println("PRODUCT: " + product + " --> PRICE: " + price);

            Thread.sleep(500);
            WebElement checkbox = driver.findElement(
                By.xpath("//table[@id='productTable']/tbody/tr[" + r + "]/td[4]/input"));

            if (!checkbox.isSelected()) {
                js.executeScript("arguments[0].click();", checkbox);
            }
        }
    }
}

```

```

        }

        Thread.sleep(1000);
    }

    System.out.println("RESULT: All products selected across all pages.");
}

// =====
// UNIT 5 - FORM
// =====

By section1Textbox = By.xpath("//input[@id='input1']");
By section1Submit = By.xpath("//button[@id='btn1']");
By section2Textbox = By.xpath("//input[@id='input2']");
By section2Submit = By.xpath("//button[@id='btn2']");
By section3Textbox = By.xpath("//input[@id='input3']");
By section3Submit = By.xpath("//button[@id='btn3']");

public void fillSection1(String text) throws InterruptedException {
    driver.findElement(section1Textbox).sendKeys(text);
    Thread.sleep(1000);
    driver.findElement(section1Submit).click();
    System.out.println("RESULT: Section 1 form submitted with text: " + text);
}

public void fillSection2(String text) throws InterruptedException {
    driver.findElement(section2Textbox).sendKeys(text);
    Thread.sleep(1000);
    driver.findElement(section2Submit).click();
    System.out.println("RESULT: Section 2 form submitted with text: " + text);
}

public void fillSection3(String text) throws InterruptedException {

```

```
driver.findElement(section3Textbox).sendKeys(text);

Thread.sleep(1000);

driver.findElement(section3Submit).click();

System.out.println("RESULT: Section 3 form submitted with text: " + text);
}
```

```
// =====
```

```
// UNIT 6 - LINKS
```

```
// =====
```

```
By allLinks = By.tagName("a");
```

```
public int getTotalLinks() {
    int count = driver.findElements(allLinks).size();
    System.out.println("RESULT: Total links found: " + count);
    return count;
}
```

```
public void printValidLinks() {
    List<WebElement> links = driver.findElements(allLinks);
    for (WebElement link : links) {
        String url = link.getAttribute("href");
        if (url != null && !url.isEmpty()) {
            System.out.println("VALID LINK: " + url);
        }
    }
    System.out.println("RESULT: All valid links printed successfully.");
}
```

```
// =====
```

```
// UNIT 7 - TABS & WIKIPEDIA
```

```
// =====
```

```
By wikiBox = By.id("Wikipedia1_wikipedia-search-input");  
By wikiBtn = By.className("wikipedia-search-button");  
By googleLink = By.xpath("//div[@id='Wikipedia1_wikipedia-search-results']/a[text()='Google']");  
By startBtn = By.xpath("//button[text()='START']");  
By stopBtn = By.xpath("//button[text()='STOP']");
```

```
public void searchWikipediaAndNavigateTab() throws InterruptedException {  
    driver.findElement(wikiBox).sendKeys("Google");  
    driver.findElement(wikiBtn).click();  
    String currentTab = driver.getWindowHandle();  
  
    Thread.sleep(2000);  
    driver.findElement(googleLink).click();  
  
    for (String handle : driver.getWindowHandles()) {  
        if (!handle.equals(currentTab)) {  
            driver.switchTo().window(handle);  
            System.out.println("RESULT: Opened Tab URL: " + driver.getCurrentUrl());  
            driver.close();  
            break;  
        }  
    }  
    driver.switchTo().window(currentTab);  
  
    driver.findElement(startBtn).click();  
    System.out.println("ACTION: Clicked START button.");  
    Thread.sleep(2000);  
  
    driver.findElement(stopBtn).click();  
    System.out.println("ACTION: Clicked STOP button.");  
    System.out.println("RESULT: Wikipedia tab navigation completed successfully.");  
}
```

```

}

// =====
// UNIT 8 - ALERTS, POPUPS & WINDOWS
// =====

By alertButton = By.id("alertBtn");
By confirmButton = By.id("confirmBtn");
By promptButton = By.id("promptBtn");
By tabButton = By.xpath("//button[text()='New Tab']");
By popUpWindowButton = By.id("PopUp");

public void executeUnit8_AlertsAndPopups() throws InterruptedException {

    String parentWindow = driver.getWindowHandle();

    JavascriptExecutor js = (JavascriptExecutor) driver;

    // Alerts

    driver.findElement(alertButton).click();

    Thread.sleep(1000);

    driver.switchTo().alert().accept();

    System.out.println("RESULT: Simple Alert accepted.");

    driver.findElement(confirmButton).click();

    Thread.sleep(1000);

    driver.switchTo().alert().dismiss();

    System.out.println("RESULT: Confirmation Alert dismissed.");

    driver.findElement(promptButton).click();

    Thread.sleep(1000);

    driver.switchTo().alert().sendKeys("Bhargav Automation");

    driver.switchTo().alert().accept();

    System.out.println("RESULT: Prompt Alert handled with input.");
}

```

```
// New Tab
```

```
js.executeScript("arguments[0].scrollIntoView(true);", driver.findElement(tabButton));
```

```
driver.findElement(tabButton).click();
```

```
Thread.sleep(2000);
```

```
for (String handle : driver.getWindowHandles()) {
```

```
    if (!handle.equals(parentWindow)) {
```

```
        driver.switchTo().window(handle);
```

```
        System.out.println("RESULT: New Tab URL: " + driver.getCurrentUrl());
```

```
        driver.close();
```

```
        break;
```

```
    }
```

```
}
```

```
driver.switchTo().window(parentWindow);
```

```
// Popup Window
```

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(5));
```

```
WebElement popupBtn =
```

```
wait.until(ExpectedConditions.elementToBeClickable(popUpWindowButton));
```

```
popupBtn.click();
```

```
Thread.sleep(2000);
```

```
Set<String> allWindows = driver.getWindowHandles();
```

```
for (String handle : allWindows) {
```

```
    if (!handle.equals(parentWindow)) {
```

```
        driver.switchTo().window(handle);
```

```
        System.out.println("RESULT: Popup Window URL: " + driver.getCurrentUrl());
```

```
        driver.close();
```

```
        break;
```

```
    }
```

```
}
```

```
driver.switchTo().window(parentWindow);
```

```

        System.out.println("RESULT: Popup Window handled successfully.");
    }

    // =====
    // UNIT 9 - ADVANCED MOUSE INTERACTIONS
    // =====

    By field1Box = By.id("field1");

    By copyTextBtn = By.xpath("//button[text()='Copy Text']");

    By dragSource = By.id("draggable");

    By dropTarget = By.id("droppable");

    By pointMeBtn = By.className("dropbtn");

    By subMenuItemLaptops = By.xpath("//div[@class='dropdown-content']/a[text()='Laptops']");

    By sliderHandle = By.cssSelector("#slider span.ui-slider-handle");

    public void executeUnit9_AdvancedMouseActions() throws InterruptedException {

        JavascriptExecutor js = (JavascriptExecutor) driver;

        org.openqa.selenium.interactions.Actions actions = new
        org.openqa.selenium.interactions.Actions(driver);

        // Hover

        js.executeScript("arguments[0].scrollIntoView({block: 'center'});",
        driver.findElement(pointMeBtn));

        Thread.sleep(1000);

        actions.moveToElement(driver.findElement(pointMeBtn)).perform();

        System.out.println("ACTION: Hovered over 'Point Me'.");

        driver.findElement(subMenuItemLaptops).click();

        System.out.println("RESULT: Clicked 'Laptops' submenu.");

        // Double Click

        js.executeScript("arguments[0].scrollIntoView({block: 'center'});",
        driver.findElement(copyTextBtn));

        driver.findElement(field1Box).clear();
    }

```

```
driver.findElement(field1Box).sendKeys("Bhargav Automation");
actions.doubleClick(driver.findElement(copyTextBtn)).perform();
System.out.println("RESULT: Double Clicked 'Copy Text'.");
```

```
// Drag & Drop
```

```
js.executeScript("arguments[0].scrollIntoView({block: 'center'});", driver.findElement(dropTarget));
actions.dragAndDrop(driver.findElement(dragSource), driver.findElement(dropTarget)).perform();
System.out.println("RESULT: Drag and Drop completed.");
```

```
// Slider
```

```
WebElement slider = driver.findElement(By.xpath("//span[@class='ui-slider-handle ui-corner-all ui-state-default']")[1]));
js.executeScript("arguments[0].scrollIntoView({block: 'center'});", slider);
actions.dragAndDropBy(slider, 50, 0).perform();
System.out.println("RESULT: Slider moved 50px right.");
```

```
// ComboBox
```

```
WebElement comboBox = driver.findElement(By.id("comboBox"));
js.executeScript("arguments[0].scrollIntoView({block: 'center'});", comboBox);
comboBox.click();
WebElement option = driver.findElement(By.xpath("//div[@class='option' and text()='Item 1']"));
option.click();
System.out.println("RESULT: ComboBox option 'Item 1' selected.");
}
```

```
// =====
```

```
// UNIT 10 - LINK TRAVERSAL VALIDATION
```

```
// =====
```

```
public void executeUnit10_LinkTraversal() throws InterruptedException {
    JavascriptExecutor js = (JavascriptExecutor) driver;
```

```
// Capture main window and base homepage URL
String mainWindowHandle = driver.getWindowHandle();
String basePageUrl = driver.getCurrentUrl();

String[] targetLinks = {
    "Apple", "Lenovo", "Dell",
    "Errorcode 400", "Errorcode 401", "Errorcode 403",
    "Errorcode 404", "Errorcode 408", "Errorcode 500",
    "Errorcode 502", "Errorcode 503"
};

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
System.out.println("STEP: Traversing " + targetLinks.length + " links...");

for (String linkText : targetLinks) {
    try {
        // Ensure focus on main window
        driver.switchTo().window(mainWindowHandle);

        // If not on base homepage, return to it
        if (!driver.getCurrentUrl().equals(basePageUrl)) {
            driver.get(basePageUrl);
            wait.until(d -> js.executeScript("return document.readyState").equals("complete"));
        }

        // Locate link using normalized text
        By linkLocator = By.xpath("//a[normalize-space()='\" + linkText + "\"]");

        WebElement currentLink =
wait.until(ExpectedConditions.visibilityOfElementLocated(linkLocator));

        // Scroll into view
```

```

js.executeScript("arguments[0].scrollIntoView({block: 'center'});", currentLink);

Thread.sleep(500);

// Click link
wait.until(ExpectedConditions.elementToBeClickable(currentLink));
currentLink.click();

// Wait for page load
wait.until(d -> js.executeScript("return document.readyState").equals("complete"));
System.out.println("RESULT: Link opened -> " + linkText);
Thread.sleep(1000);

// Handle new tab or same tab
Set<String> allWindows = driver.getWindowHandles();
if (allWindows.size() > 1) {
    for (String windowHandle : allWindows) {
        if (!windowHandle.equals(mainWindowHandle)) {
            driver.switchTo().window(windowHandle);
            driver.close();
        }
    }
    driver.switchTo().window(mainWindowHandle);
} else {
    driver.navigate().back();
    wait.until(ExpectedConditions.urlToBe(baseUrl));
    wait.until(d -> js.executeScript("return document.readyState").equals("complete"));
    Thread.sleep(500);
}

} catch (Exception e) {
    System.out.println("⚠ Skipped or timed out processing link: [" + linkText + "]");
}

```

```

        try {
            driver.switchTo().window(mainWindowHandle);

            driver.get(basePageUrl);

            wait.until(d -> js.executeScript("return document.readyState").equals("complete"));
        } catch (Exception ex) {
            // fallback
        }
    }
}

System.out.println("=== UNIT 10 COMPLETE: ALL LINKS TRAVERSED ===");
}

// =====
// UNIT 11 - SHADOW DOM
// =====

public void executeUnit11_ShadowDOM() throws InterruptedException {
    JavascriptExecutor js = (JavascriptExecutor) driver;

    // Scroll to Shadow DOM section
    WebElement shadowHost = driver.findElement(By.id("shadow_host"));
    js.executeScript("arguments[0].scrollIntoView({block:'center'});", shadowHost);
    Thread.sleep(2000);

    SearchContext shadowRoot = shadowHost.getShadowRoot();
    System.out.println("STEP: Unit 11 - Shadow DOM");

    // Print Mobile & Laptop info
    String shadowText = shadowRoot.findElement(By.cssSelector(".info")).getText();
    System.out.println("RESULT: Shadow DOM Info -> " + shadowText);
    Thread.sleep(1000);
}

```

```
// Blog Link
WebElement blogLink = shadowRoot.findElement(By.linkText("Blog"));
String blogUrl = blogLink.getAttribute("href");
String mainWindow = driver.getWindowHandle();

// Open Blog in new tab
js.executeScript("window.open(arguments[0]);", blogUrl);
Thread.sleep(3000);

String blogTitle = "";
for (String win : driver.getWindowHandles()) {
    if (!win.equals(mainWindow)) {
        driver.switchTo().window(win);
        blogTitle = driver.getTitle();
        System.out.println("RESULT: Blog Title -> " + blogTitle);
        Thread.sleep(2000);
        driver.close();
        break;
    }
}
driver.switchTo().window(mainWindow);

// Re-acquire Shadow DOM
shadowHost = driver.findElement(By.id("shadow_host"));
shadowRoot = shadowHost.getShadowRoot();

// Textbox
WebElement txtBox = shadowRoot.findElement(By.cssSelector("input[type='text']"));
txtBox.clear();
txtBox.sendKeys(blogTitle);
System.out.println("ACTION: Entered Blog Title into Shadow DOM textbox.");
```

```

Thread.sleep(1000);

// Checkbox
WebElement checkbox = shadowRoot.findElement(By.cssSelector("input[type='checkbox']"));
if (!checkbox.isSelected()) {
    checkbox.click();
    System.out.println("ACTION: Shadow DOM checkbox selected.");
}
Thread.sleep(1000);

// Upload File
WebElement upload = shadowRoot.findElement(By.cssSelector("input[type='file']"));
upload.sendKeys("C:\\Users\\Admin\\OneDrive\\Pictures\\133907427220636723.jpg");
System.out.println("RESULT: File uploaded inside Shadow DOM.");
Thread.sleep(2000);

System.out.println("=== UNIT 11 COMPLETE ===");
}

```

```

package tests;

```

```

import static org.testng.AssertJUnit.assertTrue;
import java.util.List;

import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebElement;
import org.testng.Assert;
import org.testng.annotations.AfterClass;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Listeners;
import org.testng.annotations.Test;

```

```

import base.BaseClass;
import pages.AutomationPracticePage;
import utils.WriteExcelwithdp;

@Listeners(listeners.TestListener.class)
public class AutomationPracticeTest extends BaseClass {

    AutomationPracticePage page;
    WriteExcelwithdp excel;

    // alwaysRun = true guarantees this executes before your groups run
    @BeforeClass(alwaysRun = true)
    public void startBrowser() {

        System.out.println("🚀 Initializing browser and layout configurations...");

        setup();

        page = new AutomationPracticePage(driver);

        excel = new WriteExcelwithdp();
        excel.createExcelFile();
    }
    // =====
    // COMMON SCROLL METHODS
    // =====

    public void scrollToElement(By locator)
        throws InterruptedException {

        WebElement element =
            driver.findElement(locator);

        JavascriptExecutor js =
            (JavascriptExecutor) driver;

        js.executeScript(
            "arguments[0].scrollIntoView({block:'center'});",
            element);

        Thread.sleep(1000);
    }
}

```

```

// =====
// DATAPROVIDER
// =====

@DataProvider(name="userData")
public Object[][] userData(){

    return new Object[][]{

        {
            "Bhargav",
            "bhargav@gmail.com",
            "9876543210",
            "Kakinada"
        }, {
            "pavan",
            "pavan@gmail.com",
            "9876543210",
            "mumbai"
        }

    };

}

// UNIT 1

@Test(priority = 1,
    dataProvider = "userData", groups = { "smoke" })
public void Unit1_GUIElements(
    String name,
    String email,
    String phone,
    String address) {

    page.enterName(name);

    page.enterEmail(email);

    page.enterPhone(phone);

    page.enterAddress(address);

    page.selectGender("Male");

```

```
page.selectDays();

page.selectCountry("India");

page.selectColor("Blue");

page.selectAnimal("Dog");
}
```

// UNIT 2

```
@Test(priority = 2, groups = { "regression" })
public void Unit2_DatePickers() throws InterruptedException {

    page.selectDatePicker1("06/05/2026");
    Thread.sleep(1000);
    page.selectDatePicker2("05/06/2026");
    Thread.sleep(1000);

    page.selectDateRange(
        "01-06-2026",
        "10-06-2026");
    Thread.sleep(1000);

    page.submitDateRange();
    Thread.sleep(2000);
}
```

// UNIT 3

```
@Test(priority = 3, groups = { "smoke" })
public void Unit3_FileUpload() throws InterruptedException {

    JavascriptExecutor js =
        (JavascriptExecutor) driver;

    WebElement fileUploadSection =
        driver.findElement(By.id("singleFileInput"));

    js.executeScript(
        "arguments[0].scrollIntoView({block:'center'});",
        fileUploadSection);
}
```

```

Thread.sleep(2000);

page.uploadSingleFile(
    "C:\\Users\\Admin\\OneDrive\\Pictures\\133907427220636723.jpg");

Thread.sleep(1000);

page.uploadMultipleFiles(
    "C:\\Users\\Admin\\OneDrive\\Pictures\\133907427220636723.jpg",
    "C:\\Users\\Admin\\OneDrive\\Pictures\\133907427220636723.jpg");

Thread.sleep(1000);
}
//UNIT 4
@Test(priority = 4, groups = { "regression" })
public void Unit4_PaginationWebTable()
    throws InterruptedException {

    JavascriptExecutor js =
        (JavascriptExecutor) driver;

    WebElement tableSection =
        driver.findElement(By.id("productTable"));

    js.executeScript(
        "arguments[0].scrollIntoView({block:'center'});",
        tableSection);

    Thread.sleep(2000);

    System.out.println(
        "Total Pages = "
        + page.getTotalPages());

    page.selectAllProductsInAllPages();

    Thread.sleep(1000);
}

//UNIT 5
@Test(priority = 5, groups = { "smoke" })
public void Unit5_Form()
    throws InterruptedException {

    page.fillSection1("Bhargav");

```

```

Thread.sleep(2000);

page.fillSection2("Automation Testing");
Thread.sleep(2000);
page.fillSection3("Selenium TestNG");

Thread.sleep(2000);

System.out.println(
    "Form Submitted Successfully");
}
//UNIT 6
@Test(priority = 6,groups = { "smoke" })
public void printValidLinksTest() throws InterruptedException {
    System.out.println("Executing priority 6 test - printValidLinks");
    Thread.sleep(2000);

    // call your method here
    printValidLinks();
}

public void printValidLinks() {
    System.out.println("Running link validation logic...");
}

// UNIT 7

@Test(priority = 7,groups = { "regression" })
public void Unit7_TabsAndWindowsValidation() throws InterruptedException {
    System.out.println("Executing priority 7 test - Wikipedia Search, Tab Swap, & Focus Reset");
    Thread.sleep(2000);

    // Data is managed directly inside the POM method layer
    page.searchWikipediaAndNavigateTab();

}
// UNIT 8

@Test(priority = 8, groups = { "regression" })
public void Unit8_AlertsAndPopupsValidation() throws InterruptedException {
    System.out.println("Executing priority 8 test - Alerts & Popups");
    Thread.sleep(2000);

    // Triggers the entire simple, confirm, and prompt sequential flow
    page.executeUnit8_AlertsAndPopups();
}

```

```

    }
    // UNIT 9 - ADVANCED MOUSE INTERACTIONS

    @Test(priority = 9, groups = { "smoke" })
    public void Unit9_AdvancedMouseValidation() {
        System.out.println("Executing priority 9 test - Double Click, Drag-Drop, and Slider Actions");

        // Triggers the copy text, canvas box drag, and slider bar drag tasks sequentially
        page.executeUnit9_AdvancedMouseActions();
    }

    // UNIT 10 - LINK TRAVERSAL VALIDATION

    @Test(priority = 10, groups = { "regression" })
    public void Unit10_VerifyLaptopAndBrokenLinks() {
        System.out.println("Executing priority 10 test - Physical Traversal for Laptop & Error Code Links");

        // Triggers the targeted click-print-return cycle for Apple, Lenovo, Dell and the error links
        page.executeUnit10_LinkTraversal();
    }

    //UNIT 11
    @Test(priority = 11,
        groups = {"regression"})
    public void Unit11_ShadowDOM()
        throws InterruptedException {

        page.executeUnit11_ShadowDOM();
    }

    @AfterMethod
    public void afterMethod() {
        driver.quit();
    }
}

```

ExtentManager.Java

```

package utils;

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;

public class ExtentManager {

```

```

// Singleton instance
private static ExtentReports extent;

// Get ExtentReports object
public static ExtentReports getReport() {

    if (extent == null) {
        // Reporter setup
        ExtentSparkReporter spark = new ExtentSparkReporter("test-output/ExtentReport.html");

        // Report configuration
        spark.config().setReportName("Automation Practice Report");
        spark.config().setDocumentTitle("Selenium Automation Report");

        // Attach reporter
        extent = new ExtentReports();
        extent.attachReporter(spark);

        // System info
        extent.setSystemInfo("Tester", "Bhargav");
        extent.setSystemInfo("Framework", "Selenium TestNG");
    }

    return extent;
}
}

```

WriteExcelwithdp.Java

```

package utils;

import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.text.SimpleDateFormat;
import java.util.Date;

import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public class WriteExcelwithdp {

```

```

private final String path = "C:\\Users\\Admin\\exxc1\\TestData123.xlsx";

// ===== CREATE EXCEL FILE =====
public void createExcelFile() {
    File file = new File(path);
    try {
        if (!file.getParentFile().exists()) {
            file.getParentFile().mkdirs();
        }

        XSSFWorkbook workbook = null;

        // Safe load check
        if (file.exists()) {
            try (FileInputStream fis = new FileInputStream(file)) {
                workbook = new XSSFWorkbook(fis);
            } catch (Exception e) {
                System.err.println("⚠ Existing Excel file corrupted. Resetting...");
                workbook = null;
            }
        }

        if (workbook == null) {
            workbook = new XSSFWorkbook();
        }

        // Sheets setup
        String[] sheets = {"FormInputsData", "SmokeSuite", "RegressionSuite"};
        for (String sheetName : sheets) {
            if (workbook.getSheet(sheetName) == null) {
                Sheet sheet = workbook.createSheet(sheetName);
                Row header = sheet.createRow(0);

                if (sheetName.endsWith("Suite")) {
                    header.createCell(0).setCellValue("Test Case / Scenario Name");
                    header.createCell(1).setCellValue("Status");
                    header.createCell(2).setCellValue("Execution Timestamp");
                } else if (sheetName.equals("FormInputsData")) {
                    header.createCell(0).setCellValue("Name");
                    header.createCell(1).setCellValue("Email");
                    header.createCell(2).setCellValue("Phone");
                }
            }
        }
    }
}

```

```

        header.createCell(3).setCellValue("Address");

        Row dataRow = sheet.createRow(1);
        dataRow.createCell(0).setCellValue("Bhargav");
        dataRow.createCell(1).setCellValue("bhargav@gmail.com");
        dataRow.createCell(2).setCellValue("9876543210");
        dataRow.createCell(3).setCellValue("Kakinada");

        Row row2 = sheet.createRow(2);
        row2.createCell(0).setCellValue("pavan");
        row2.createCell(1).setCellValue("pavan@gmail.com");
        row2.createCell(2).setCellValue("9876543210");
        row2.createCell(3).setCellValue("mumbai");
    }
}

try (FileOutputStream fos = new FileOutputStream(file)) {
    workbook.write(fos);
    fos.flush();
}
workbook.close();
System.out.println("✅ Excel File initialized at: " + path);

} catch (Exception e) {
    System.err.println("❌ Failed to verify excel path: " + e.getMessage());
    e.printStackTrace();
}
}

// ===== READ INPUT MATRIX =====
public Object[][] readInputMatrix(String sheetName) {
    Object[][] data = null;
    File file = new File(path);
    if (!file.exists()) return new Object[0][0];

    try (FileInputStream fis = new FileInputStream(file);
        Workbook workbook = new XSSFWorkbook(fis)) {

        Sheet sheet = workbook.getSheet(sheetName);
        if (sheet == null) return new Object[0][0];
    }
}

```

```

int rowCount = sheet.getLastRowNum();
if (rowCount < 1) return new Object[0][0];

int colCount = sheet.getRow(0).getLastCellNum();
data = new Object[rowCount][colCount];
DataFormatter formatter = new DataFormatter();

for (int i = 1; i <= rowCount; i++) {
    Row row = sheet.getRow(i);
    for (int j = 0; j < colCount; j++) {
        if (row == null) {
            data[i - 1][j] = "";
        } else {
            Cell cell = row.getCell(j);
            data[i - 1][j] = formatter.formatCellValue(cell).trim();
        }
    }
}
} catch (Exception e) {
    System.err.println("✗ Error reading data matrix: " + e.getMessage());
}
return data;
}

// ===== LOG SUITE RESULT =====
public synchronized void logSuiteResult(String suiteSheetName, String testName, String
status) {
    File file = new File(path);
    XSSFWorkbook workbook = null;

    try (FileInputStream fis = new FileInputStream(file)) {
        workbook = new XSSFWorkbook(fis);
    } catch (Exception e) {
        System.err.println("✗ Error opening Excel for logging: " + e.getMessage());
        return;
    }

    try {
        Sheet sheet = workbook.getSheet(suiteSheetName);
        if (sheet == null) {

```

```

        sheet = workbook.createSheet(suiteSheetName);
    }

    int nextRowIndex = sheet.getPhysicalNumberOfRows();
    if (nextRowIndex == 0) {
        Row header = sheet.createRow(0);
        header.createCell(0).setCellValue("Test Case / Scenario Name");
        header.createCell(1).setCellValue("Status");
        header.createCell(2).setCellValue("Execution Timestamp");
        nextRowIndex = 1;
    }

    Row row = sheet.createRow(nextRowIndex);
    String timeStampStr = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(new
Date());

    row.createCell(0).setCellValue(testName);
    row.createCell(1).setCellValue(status);
    row.createCell(2).setCellValue(timeStampStr);

    try (FileOutputStream fos = new FileOutputStream(file)) {
        workbook.write(fos);
        fos.flush();
    }
    workbook.close();
    System.out.println("🔴 Logged result -> Sheet: [" + suiteSheetName + "] Row: " +
nextRowIndex);

    } catch (Exception e) {
        System.err.println("❌ Error appending data: " + e.getMessage());
    }
}
}

```

TestListener.Java

```

package listeners;

import org.testng.ITestContext;
import org.testng.ITestListener;
import org.testng.ITestResult;

```

```

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;

import utils.ExtentManager;
import utils.WriteExcelwithdp;

public class TestListener implements ITestListener {

    private final WriteExcelwithdp excel = new WriteExcelwithdp();
    private final ExtentReports extent = ExtentManager.getReport();
    private ExtentTest test;

    @Override
    public void onStart(ITestContext context) {
        System.out.println("\n=====");
        System.out.println(" AUTOMATION EXECUTION STARTED");
        System.out.println("=====\\n");
        excel.createExcelFile();
    }

    @Override
    public void onTestStart(ITestResult result) {
        test = extent.createTest(result.getMethod().getMethodName());

        System.out.println("\n=====");
        System.out.println("TEST CASE : " + result.getMethod().getMethodName());
        System.out.println("STATUS   : RUNNING");
        System.out.println("=====");
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        test.pass("Test Passed");

        System.out.println("=====");
        System.out.println("TEST CASE : " + result.getMethod().getMethodName());
        System.out.println("STATUS   : PASSED");
        System.out.println("=====");
    }

```

```

        routeToExcelSuite(result, "PASSED");
    }

    @Override
    public void onTestFailure(ITestResult result) {
        String reason = (result.getThrowable() != null)
            ? result.getThrowable().getMessage()
            : "Unknown Error";

        test.fail(reason);

        System.out.println("=====");
        System.out.println("TEST CASE : " + result.getMethod().getMethodName());
        System.out.println("STATUS   : FAILED");
        System.out.println("REASON   : " + reason);
        System.out.println("=====");

        routeToExcelSuite(result, "FAILED");
    }

    @Override
    public void onTestSkipped(ITestResult result) {
        test.skip("Test Skipped");

        System.out.println("=====");
        System.out.println("TEST CASE : " + result.getMethod().getMethodName());
        System.out.println("STATUS   : SKIPPED");
        System.out.println("=====");
    }

    @Override
    public void onFinish(ITestContext context) {
        extent.flush();

        System.out.println("\n=====");
        System.out.println(" AUTOMATION EXECUTION COMPLETED");
        System.out.println("=====");
        System.out.println("\n Extent Report Generated:");
        System.out.println("test-output\\ExtentReport.html");
    }
}

```

```

private void routeToExcelSuite(ITestResult result, String status) {
    String testName = result.getMethod().getMethodName();
    String[] groups = result.getMethod().getGroups();

    boolean isSmoke = false;
    boolean isRegression = false;

    for (String group : groups) {
        if (group.equalsIgnoreCase("smoke")) {
            isSmoke = true;
        }
        if (group.equalsIgnoreCase("regression")) {
            isRegression = true;
        }
    }

    if (isSmoke) {
        excel.logSuiteResult("SmokeSuite", testName, status);
    }
    if (isRegression) {
        excel.logSuiteResult("RegressionSuite", testName, status);
    }
}

```

POM.XML

```

<project
xmlns="http://maven.apache.org/POM/4.0.0"xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd>
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.automation</groupId>
    <artifactId>AutomationPracticeFramework</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <dependencies>

    <!-- Selenium -->

```

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.21.0</version>
</dependency>
```

```
<!-- TestNG -->
```

```
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.11.0</version>
  <scope>test</scope>
</dependency>
```

```
</dependency>
```

```
<dependency>
<groupId>com.aventstack</groupId>
<artifactId>extentreports</artifactId>
<version>5.1.1</version>
```

```
</dependency>
```

```
<!-- WebDriverManager -->
```

```
<dependency>
  <groupId>io.github.bonigarcia</groupId>
  <artifactId>webdrivermanager</artifactId>
  <version>5.9.2</version>
</dependency>
```

```
<!-- Apache POI Excel -->
```

```
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi</artifactId>
  <version>5.4.1</version>
</dependency>
```

```
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.4.1</version>
</dependency>
```

```
<!-- Extent Reports -->
```

```
<dependency>
  <groupId>com.aventstack</groupId>
  <artifactId>extentreports</artifactId>
  <version>5.1.2</version>
</dependency>

<!-- Log4j -->
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-api</artifactId>
  <version>2.24.3</version>
</dependency>

<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.24.3</version>
</dependency>

</dependencies>
<build>
  <plugins>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.13.0</version>
      <configuration>
        <source>17</source>
        <target>17</target>
      </configuration>
    </plugin>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.5.2</version>
    </plugin>

  </plugins>
</build>
```

```
</project>
```

Testing.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
```

```
<suite name="AutomationPracticeSuite">
```

```
  <listeners>
```

```
    <listener class-name="listeners.TestListener"/>
```

```
  </listeners>
```

```
  <test name="AllTests">
```

```
    <classes>
```

```
      <class name="tests.AutomationPracticeTest"/>
```

```
    </classes>
```

```
  </test>
```

```
</suite>
```

smoke.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
```

```
<suite name="SmokeSuite">
```

```
  <listeners>
```

```
    <listener class-name="listeners.TestListener"/>
```

```
  </listeners>
```

```
  <test name="SmokeTests">
```

```
    <groups>
```

```
      <run>
```

```
        <include name="smoke"/>
```

```
      </run>
```

```
    </groups>
```

```
    <classes>
```

```
      <class name="tests.AutomationPracticeTest"/>
```

```
    </classes>
```

```
  </test>
```

```
</suite>
```

regression.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
```

```
<suite name="RegressionSuite" verbose="0">
```

```
  <listeners>
```

```
    <listener class-name="listeners.TestListener"/>
```

```
  </listeners>
```

```
  <test name="RegressionTests">
```

```
    <groups>
```

```
      <run>
```

```
        <include name="regression"/>
```

```
      </run>
```

```
    </groups>
```

```
    <classes>
```

```
      <class name="tests.AutomationPracticeTest"/>
```

```
    </classes>
```

```
  </test>
```

```
</suite>
```

Jenkinsfile

```
pipeline {
```

```
  agent any
```

```
  stages {
```

```
    stage('Check Environment') {
```

```
      steps {
```

```
        bat 'java -version'
```

```
        bat 'mvn -version'
```

```
      }
```

```
    }
```

```
    stage('Clean Project') {
```

```
      steps {
```

```

        bat 'mvn clean'
    }
}

stage('Run Tests') {
    steps {
        bat 'mvn test'
    }
}

post {
    always {
        echo 'Execution Completed'
    }

    success {
        echo 'Build Successful'
    }

    failure {
        echo 'Build Failed'
    }
}
}

```

Output:

```

=====
AUTOMATION EXECUTION STARTED
=====

```

✅ Master Excel File verified/initialized safely at: C:\Users\Admin\exxccl1\TestData123.xlsx

Jun 14, 2026 5:05:06 PM org.testng.log4testng.Logger info

INFO: [Utils] DynamicGraphHelper.createDynamicGraph() took 15 ms.

🚀 Initializing browser and layout configurations...

Jun 14, 2026 5:05:07 PM io.github.bonigarcia.wdm.WebDriverManager resolveDriverVersion

INFO: Using chromedriver 149.0.7827.115 (resolved driver for Chrome 149)

Jun 14, 2026 5:05:07 PM io.github.bonigarcia.wdm.WebDriverManager exportDriver

INFO: Exporting webdriver.chrome.driver as

C:\Users\Admin\.cache\selenium\chromedriver\win64\149.0.7827.115\chromedriver.exe

Jun 14, 2026 5:05:10 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch

WARNING: Unable to find CDP implementation matching 149

Jun 14, 2026 5:05:10 PM org.openqa.selenium.chromium.ChromiumDriver lambda\$new\$3

WARNING: Unable to find version of CDP to use for 149.0.7827.114. You may need to include a dependency on a specific version of the CDP using something similar to `org.seleniumhq.selenium:selenium-devtools-v86:4.43.0` where the version ("v86") matches the version of the chromium-based browser you're using and the version number of the artifact is the same as Selenium's.

✅ Master Excel File verified/initialized safely at: C:\Users\Admin\exxcel1\TestData123.xlsx

=====

TEST CASE : Unit1_GUIElements

STATUS : RUNNING

=====

=====

TEST CASE : Unit1_GUIElements

STATUS : PASSED

=====

📌 Saved to Excel -> Sheet: [SmokeSuite] at Row: 1 for data profile.

=====

TEST CASE : Unit1_GUIElements

STATUS : RUNNING

=====

=====

TEST CASE : Unit1_GUIElements

STATUS : PASSED

=====

📌 Saved to Excel -> Sheet: [SmokeSuite] at Row: 2 for data profile.

=====

TEST CASE : Unit2_DatePickers

STATUS : RUNNING

=====

=====

TEST CASE : Unit2_DatePickers

STATUS : PASSED

=====

📌 Saved to Excel -> Sheet: [RegressionSuite] at Row: 1 for data profile.

=====

TEST CASE : Unit3_FileUpload

STATUS : RUNNING

TEST CASE : Unit3_FileUpload

STATUS : PASSED

📌 Saved to Excel -> Sheet: [SmokeSuite] at Row: 3 for data profile.

TEST CASE : Unit4_PaginationWebTable

STATUS : RUNNING

Total Pages = 4

Smartphone --> \$10.99

Laptop --> \$19.99

Tablet --> \$5.99

Smartwatch --> \$7.99

Wireless Earbuds --> \$8.99

Bluetooth Speaker --> \$9.99

Television --> \$20.99

Action Camera --> \$15.99

Gaming Console --> \$5.99

Digital Camera --> \$16.99

Smart Home Hub --> \$20.99

Router --> \$24.99

Portable Charger --> \$30.99

Fitness Tracker --> \$19.99

Desktop Computer --> \$2.99

E-Reader --> \$10.99

VR Headset --> \$11.99

Streaming Device --> \$13.99

Soundbar --> \$16.99

Wireless Mouse 20 --> \$17.99

TEST CASE : Unit4_PaginationWebTable

STATUS : PASSED

📌 Saved to Excel -> Sheet: [RegressionSuite] at Row: 2 for data profile.

TEST CASE : Unit5_Form

STATUS : RUNNING

Form Submitted Successfully

TEST CASE : Unit5_Form

STATUS : PASSED

📌 Saved to Excel -> Sheet: [SmokeSuite] at Row: 4 for data profile.

TEST CASE : printValidLinksTest

STATUS : RUNNING

Executing priority 6 test - printValidLinks

Running link validation logic...

TEST CASE : printValidLinksTest

STATUS : PASSED

📌 Saved to Excel -> Sheet: [SmokeSuite] at Row: 5 for data profile.

TEST CASE : Unit7_TabsAndWindowsValidation

STATUS : RUNNING

Executing priority 7 test - Wikipedia Search, Tab Swap, & Focus Reset

Opened Tab URL: <https://en.wikipedia.org/wiki/Google>

Clicked START button successfully.

Clicked STOP button successfully.

TEST CASE : Unit7_TabsAndWindowsValidation

STATUS : PASSED

📌 Saved to Excel -> Sheet: [RegressionSuite] at Row: 3 for data profile.

TEST CASE : Unit8_AlertsAndPopupsValidation

STATUS : RUNNING

Executing priority 8 test - Alerts & Popups

New Tab URL: <https://www.pavantestingtools.com/>

Popup Window URL: <https://playwright.dev/>
Popup Window handled successfully, returned to parent.

=====

TEST CASE : Unit8_AlertsAndPopupsValidation
STATUS : PASSED

=====

🚀 Saved to Excel -> Sheet: [RegressionSuite] at Row: 4 for data profile.

=====

TEST CASE : Unit9_AdvancedMouseValidation
STATUS : RUNNING

=====

Executing priority 9 test - Double Click, Drag-Drop, and Slider Actions
Hovered over Point Me button successfully.
Clicked sub-menu item 'Laptops' successfully.
Double Clicked 'Copy Text' successfully.
Drag and Drop operation completed successfully.
Slider handle dragged 50px right successfully.
ComboBox option 'Item 1' selected successfully using class properties.

=====

TEST CASE : Unit9_AdvancedMouseValidation
STATUS : PASSED

=====

🚀 Saved to Excel -> Sheet: [SmokeSuite] at Row: 6 for data profile.

=====

TEST CASE : Unit10_VerifyLaptopAndBrokenLinks
STATUS : RUNNING

=====

Executing priority 10 test - Physical Traversal for Laptop & Error Code Links

🚀 Processing 11 links dynamically via Page Load Waits...

- 🕒 link opened: Apple
- 🕒 link opened: Lenovo
- 🕒 link opened: Dell
- 🕒 link opened: Errorcode 400
- 🕒 link opened: Errorcode 401
- 🕒 link opened: Errorcode 403
- 🕒 link opened: Errorcode 404
- 🕒 link opened: Errorcode 408
- 🕒 link opened: Errorcode 500

🔗 link opened: Errorcode 502

🔗 link opened: Errorcode 503

=== UNIT 10 COMPLETE: ALL LINKS TRAVERSED SILENTLY ===

=====

TEST CASE : Unit10_VerifyLaptopAndBrokenLinks

STATUS : PASSED

=====

📌 Saved to Excel -> Sheet: [RegressionSuite] at Row: 5 for data profile.

=====

TEST CASE : Unit11_ShadowDOM

STATUS : RUNNING

=====

===== UNIT 11 =====

Mobiles

Blog Title : SDET-QA Blog

===== UNIT 11 COMPLETED =====

=====

TEST CASE : Unit11_ShadowDOM

STATUS : PASSED

=====

📌 Saved to Excel -> Sheet: [RegressionSuite] at Row: 6 for data profile.

PASSED: tests.AutomationPracticeTest.Unit9_AdvancedMouseValidation

PASSED: tests.AutomationPracticeTest.Unit7_TabsAndWindowsValidation

PASSED: tests.AutomationPracticeTest.Unit1_GUIElements("Bhargav",
"bhargav@gmail.com", "9876543210", "Kakinada")

PASSED: tests.AutomationPracticeTest.Unit1_GUIElements("pavan", "pavan@gmail.com",
"9876543210", "mumbai")

PASSED: tests.AutomationPracticeTest.Unit8_AlertsAndPopupsValidation

PASSED: tests.AutomationPracticeTest.Unit4_PaginationWebTable

PASSED: tests.AutomationPracticeTest.Unit11_ShadowDOM

PASSED: tests.AutomationPracticeTest.Unit5_Form

PASSED: tests.AutomationPracticeTest.Unit10_VerifyLaptopAndBrokenLinks

PASSED: tests.AutomationPracticeTest.Unit2_DatePickers

PASSED: tests.AutomationPracticeTest.printValidLinksTest

PASSED: tests.AutomationPracticeTest.Unit3_FileUpload

=====

Default test

Tests run: 11, Failures: 0, Skips: 0

=====

=====
AUTOMATION EXECUTION COMPLETED
=====

 Extent Report Generated:
test-output\ExtentReport.html

=====
Default suite
Total tests run: 12, Passes: 12, Failures: 0, Skips: 0
=====